

SMART RESTAURANT OCCUPANCY SYSTEM

Project Report

Course: Python for Beginners

Institution: VIT Bhopal University

Branch: CSE Core **Batch:** 2026

Student Name	RUDRAKSH KAUSHIK
Registration No.	26BCE11258
Faculty / Mentor	MANOJ KUMAR
Submission Date	30 SEPTEMBER 2026

A Python-based console application for managing restaurant occupancy, waiting queues, and estimated waiting time.

1. Introduction

The Smart Restaurant Occupancy System is a Python-based project designed around a common dining problem: a customer may know how far a restaurant is, but may not know how crowded it is or how long they may need to wait for a table.

The system maintains the current number of occupied seats, the number of waiting groups, and an estimated waiting time. It also provides a simple customer-decision calculation using travel time and estimated restaurant waiting time.

The current implementation is intentionally console-based and uses beginner-level Python concepts. The input buttons or camera layer are not implemented in this version; occupancy and queue values are entered through the program menu. This keeps the project aligned with the Python for Beginners course while leaving a clear path for future sensor or computer-vision integration.

The project follows the VITyarthi requirement that the project should identify a meaningful problem, design a technical solution, implement it using concepts learned in the course, and demonstrate the result through documentation and evaluation.

2. Problem Statement

Restaurant customers often make decisions using only distance or opening status. A restaurant that is close may still require a long wait, while a farther restaurant may have seats available immediately. There is therefore a need for a simple system that combines restaurant occupancy, waiting queue information, and estimated waiting time into one usable view.

Problem: There is no simple beginner-level system in this project context that records restaurant occupancy and waiting groups and converts those values into a useful waiting-time estimate for customers.

Proposed solution: Build a modular Python application that accepts occupancy and queue inputs, validates them, stores the current state, estimates waiting time, and calculates total customer time by combining travel time with expected waiting time.

3. Objectives

- Track the number of occupied seats in a restaurant.
- Track groups waiting outside the restaurant.
- Estimate waiting time using the current queue and available seating information.
- Allow a customer to combine travel time and estimated waiting time.
- Validate inputs so occupancy and queue values remain within logical limits.
- Store the restaurant state in a JSON file.
- Keep the implementation modular so future sensor or camera input can replace manual input.

4. Functional Requirements

The system contains more than the minimum three major functional modules required for the project.

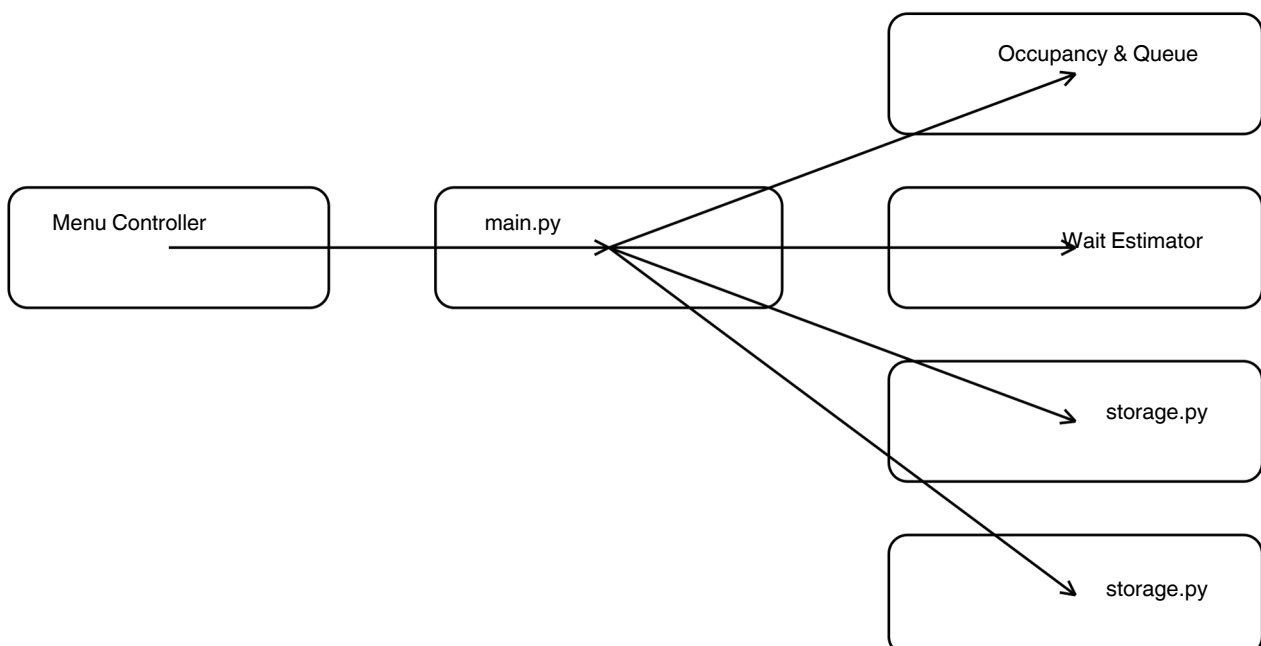
Module	Function	Input	Output
Occupancy Management	Updates current occupied seats	Person entered / left / direct	Validated occupancy
Queue Management	Adds or removes waiting groups	Group count	Validated queue size
Wait-Time Estimation	Calculates expected waiting time	Queue, capacity, occupancy	Estimated minutes
Customer Decision	Combines travel and waiting time	Travel time + wait time	Total expected time
Storage	Saves and loads state	Restaurant data	JSON file / restored state
Testing	Checks core logic	Test cases	Pass/fail results

5. Non-Functional Requirements

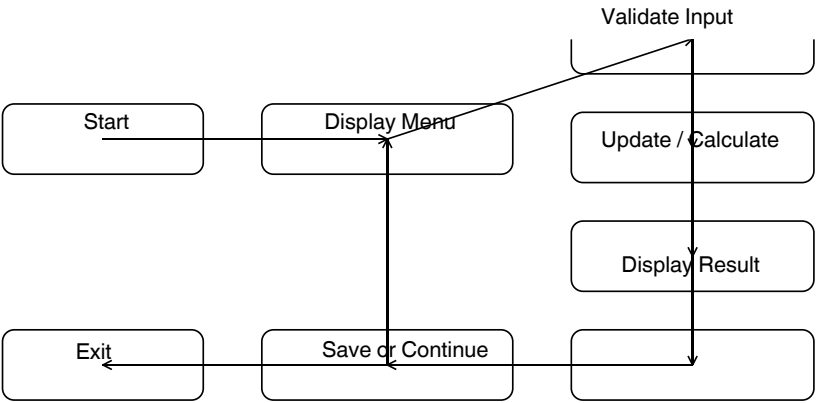
- **Usability:** The console menu uses numbered options and clear prompts so a beginner can operate the system.
- **Reliability:** Occupancy cannot become negative or exceed restaurant capacity, and queue values are validated.
- **Maintainability:** Core functions are separated into meaningful Python files instead of being placed in one large script.
- **Error Handling:** Invalid numerical values and invalid menu selections are handled through input validation.
- **Resource Efficiency:** The current version uses standard Python and a lightweight JSON file rather than a heavy database.
- **Scalability:** The design allows future replacement of manual inputs with sensors or computer vision.

SYSTEM ARCHITECTURE

The system follows a simple modular architecture. The main program handles user interaction and calls separate modules for restaurant state, occupancy, queue, waiting-time estimation, and storage.

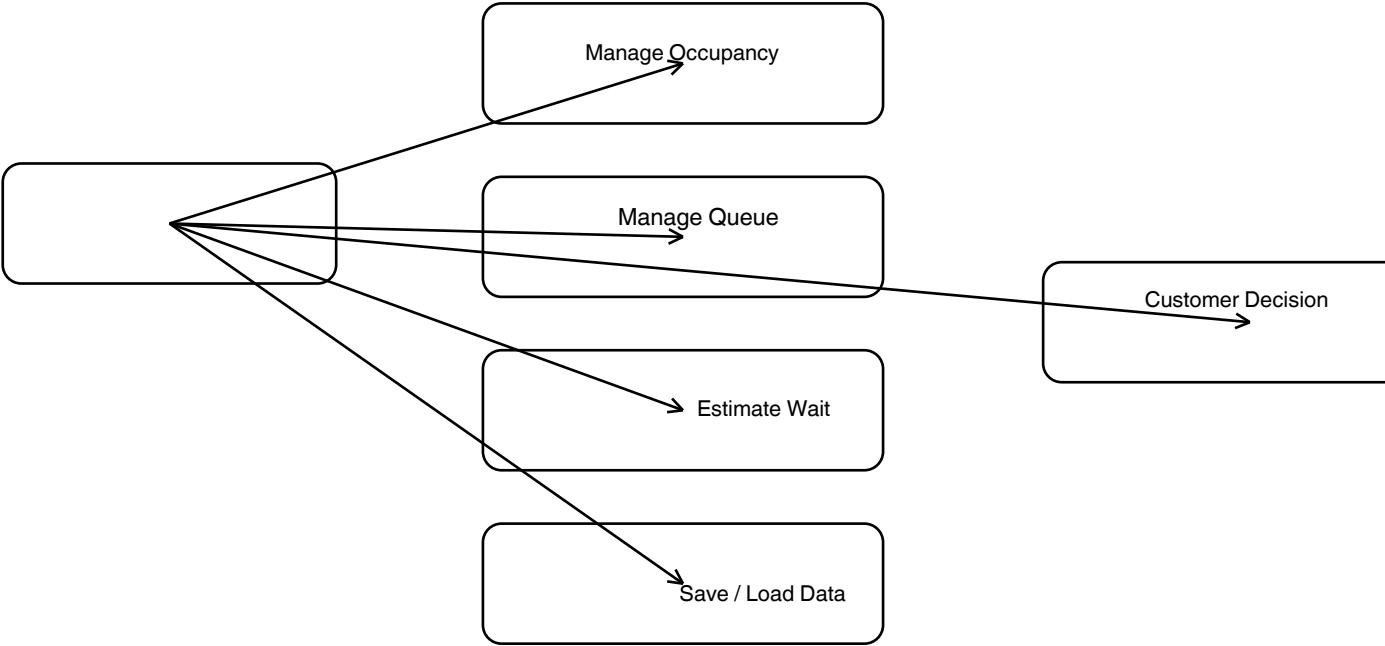


7. Process Flow / Workflow



8. Use Case Diagram

The main actor is the restaurant operator or user. The customer-facing calculation is also represented as a system operation because the current project is a console prototype rather than a public mobile application.



9. Class / Component Design

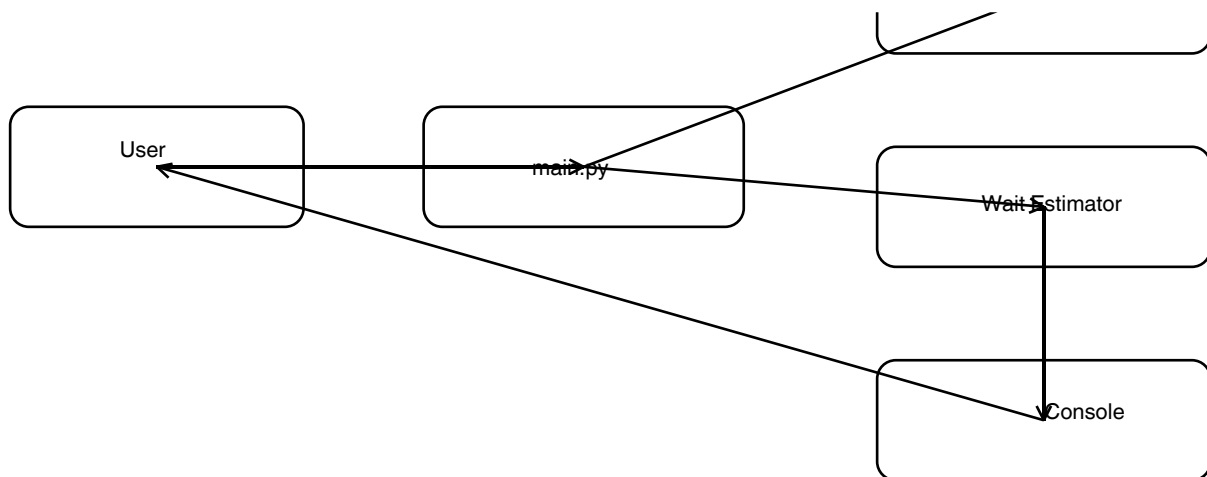
The implementation is divided into small components. The core restaurant state contains capacity, occupied seats, and queue groups. Separate managers handle occupancy and queue operations, while the wait estimator performs the calculation and the storage module handles persistence.

Component	Responsibility
Restaurant	Stores restaurant capacity, occupancy and queue state.
OccupancyManager	Validates and updates occupied-seat count.
QueueManager	Adds, removes and validates waiting groups.
WaitTimeEstimator	Calculates estimated waiting time.
Storage	Saves and restores state using JSON.

Main	Provides the console workflow and connects components.
------	--

10. Sequence Diagram

Example sequence for estimating waiting time: the user selects the estimate option, the main controller requests current state, the estimator calculates the value, and the result is displayed.



11. Storage Design

The current prototype uses a lightweight JSON file for persistence. This is sufficient for a beginner-level local project and avoids unnecessary database complexity.

Field	Type	Purpose
name	String	Restaurant name
capacity	Integer	Maximum seating capacity
occupied	Integer	Current occupied seats
queue_groups	Integer	Number of waiting groups

12. Design Decisions & Rationale

- **Console instead of GUI:** keeps the project focused on Python fundamentals and makes the logic easy to inspect.
- **Modular Python files:** separates responsibilities and supports maintainability.
- **JSON storage:** simple, readable, and suitable for a small local prototype.
- **Rule-based wait estimation:** appropriate for the current course level and avoids requiring machine-learning data.
- **Future input layer:** manual occupancy/queue updates can later be replaced by sensors or computer vision.

13. Implementation Details

The implementation uses standard Python features such as variables, functions, classes, conditional statements, loops, input handling, file handling, JSON serialization, and unit testing.

A simplified project structure is shown below:

```
Smart_Restaurant_Console_Project/  
■■■ main.py  
■■■ restaurant.py  
■■■ storage.py  
■■■ test_project.py  
■■■ requirements.txt  
■■■ README.md  
■■■ statement.md  
■■■ requirements.md  
■■■ architecture.md  
■■■ design_and_workflow.md  
■■■ testing.md  
■■■ report.md
```

The main workflow is menu-driven. The user selects an operation, enters the required value, the system validates it, updates the state or performs a calculation, and then displays the result.

14. Screenshots / Results

The current implementation produces a console menu similar to the following. This is a representative result layout; the final submission should include screenshots captured from the student's own run of the program.

SMART RESTAURANT OCCUPANCY SYSTEM
1. Person Entered
2. Person Left
3. Add Queue Group
4. Remove Queue Group
5. Show Restaurant Status
6. Estimate Waiting Time
7. Customer Decision
8. Save Data
9. Reset Data
0. Exit
Enter your choice:

Expected result: the program should update occupancy and queue values, calculate estimated waiting time, and save the state when requested.

15. Testing Approach

The project includes unit tests for the most important validation and calculation behavior.

Test	Expected behavior
Occupancy upper limit	Occupied seats cannot exceed capacity.
Occupancy lower limit	Occupied seats cannot become negative.

Queue update	Queue groups increase/decrease correctly.
Empty queue wait	Waiting time is zero when there is no waiting queue.

16. Challenges Faced

- Designing a project that is useful while still remaining appropriate for a beginner Python course.
- Keeping occupancy values within realistic limits.
- Turning queue and occupancy values into a simple waiting-time estimate.
- Keeping the code modular instead of placing all functionality in one file.
- Designing the project so future automation can be added without rewriting the core logic.

17. Learnings & Key Takeaways

- Breaking a real-world problem into smaller software modules.
- Using Python functions and classes to separate responsibilities.
- Handling invalid user input and maintaining valid system state.
- Working with JSON files for simple data persistence.
- Writing basic unit tests for important program behavior.
- Thinking about software architecture before adding advanced features.

18. Future Enhancements

- Camera-based people counting using computer vision.
- Automatic detection of the outside waiting queue.
- Real-time occupancy updates from sensors or cameras.
- A web or mobile interface for customers.
- Restaurant comparison using travel time and expected waiting time.
- Historical data analysis to improve waiting-time estimates.
- Multiple restaurants connected to one customer-facing system.

19. Conclusion

The Smart Restaurant Occupancy System demonstrates how a practical restaurant problem can be converted into a modular Python application. The current version focuses on reliable occupancy and queue management, waiting-time estimation, simple customer decision support, and local storage. Its architecture also leaves room for future automation through sensors or computer vision.

The project therefore provides a manageable foundation for a beginner Python course while addressing the major functional, non-functional, implementation, testing, and documentation expectations of the VITyarthi project format.

20. References

- VITyarthi–Build Your Own Project: General Project Instructions & Submission Guidelines.
- Python standard library concepts: functions, classes, JSON file handling, input validation, and unittest.
- Project source code and project documentation maintained in the accompanying GitHub repository.